

How I designed the overflowed buffer

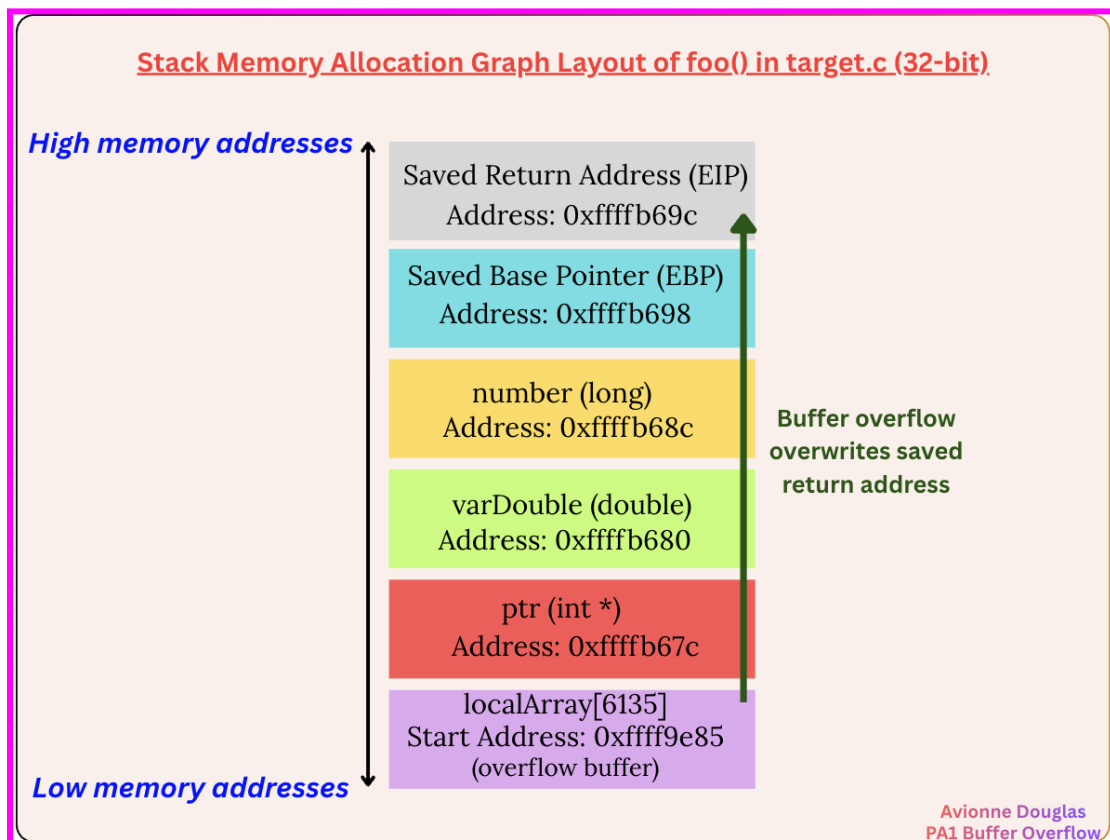
To design the overflow buffer for this assignment, I used the provided ZIP file from the webcourse and extracted it on the UCF department Eustis Linux machine. The extracted file was composed of two directories: exploits and targets. In the targets directory, I compiled and ran the target.c code, which gave me the vulnerable target executable. From there, I was able to identify that the foo() function within the target contained the overflow vulnerability. This means that when the exploit program runs on the target, it passes the buffer I created to it and is now vulnerable to a buffer overflow attack.

I created the buffer overflow by first determining the size of the localArray in the foo() function, checking the given code. Once I found out the size 6135 bytes, I made sure that the shellcode was inside the buffer to ensure it would be stored in memory when passed to the target program. Since the buffer size was much larger than the localArray, there was an overflow, and it overwrote the saved return address (EIP). This resulted in the program jumping since the foo() function could not return to its original location because the return address had been overwritten.

The foo() function then allows data to be copied into the localArray buffer. This works because the foo() function did not check to see if the bounds were followed in the inserted data. Since variables like number, localArray, varDouble, and ptr, along with the saved return address and saved base pointer, are within the stack frame after foo() is called, the buffer overflow was able to overwrite memory at a higher memory address.

After creating the buffer overflow, I followed the assignment instructions to make sure that the shellcode was executed in the proper directory. So now, when the foo() function returns, instead of returning to the given address, it is diverted to the overwritten return address, resulting in the execution of the shellcode instead of carrying out the operation normally.

Stack memory allocation graph



How I used Gdb in Eustis to find out the stack information (screenshot)

```
[av662490@net1547:~/CAP6135/PA1/exploits$ setarch i686 -R gdb ./exploit
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./exploit...
(gdb) set follow-fork-mode child
(gdb) break target.c:foo
No source file named target.c.
[Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (target.c:foo) pending.
(gdb) run
Starting program: /home/net/av662490/CAP6135/PA1/exploits/exploit

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
[Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
process 1836872 is executing new program: /home/net/av662490/CAP6135/PA1/targets/target
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Press return to call the fuction foo()...
```

Starting GDB on exploit with ASLR disabled

Enable child process debugging so breakpoint in foo() can be reached

Set breakpoint at vulnerable function foo()

GDB running exploit program

```
Breakpoint 1, foo (arg=0xffffb8ae '\220' <repeats 200 times>...) at target.c:7
7      long number = 3333344;
(gdb)
(gdb) p &localArray
$1 = (char (*)[6135]) 0xffff9e85
(gdb) p &number
$2 = (long *) 0xffffb68c
(gdb) p &varDouble
$3 = (double *) 0xffffb680
(gdb) p &ptr
$4 = (int **) 0xffffb67c
(gdb) info frame
Stack level 0, frame at 0xffffb6a0:
 eip = 0x565561ec in foo (target.c:7); saved eip = 0x5655634e
 called by frame at 0xffffb6e0
 source language c.
 Arglist at 0xffffb698, args: arg=0xffffb8ae '\220' <repeats 200 times>..
 Locals at 0xffffb698, Previous frame's sp is 0xffffb6a0
 Saved registers:
  ebx at 0xffffb694, ebp at 0xffffb698, eip at 0xffffb69c
(gdb)
```

Breakpoint hit at function foo() in target.c

Addresses of local variables stored on the stack

Stack frame information showing saved return address (EIP), base frame pointer (EBP), and stack layout

Screenshot of successful exploit that creates a shell in compromise

```
[av662490@net1547:~/CAP6135/PA1/exploits$ setarch i686 -R ./exploit
Press return to call the fuction foo()...

foo() finishes normally.
[$ $ ls
README  exploit  exploit.c  shellcode.h  testshellcode  testshellcode.c
[$ exit
av662490@net1547:~/CAP6135/PA1/exploits$
```